

Forward Proxy vs Reverse Proxy

Advanced Technical Documentation

1. Overview

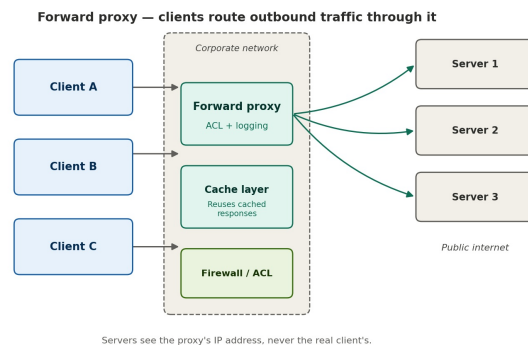
A proxy server is an intermediary that relays traffic between two parties instead of letting them talk directly. The distinction between a **forward proxy** and a **reverse proxy** is not about the technology — the same software (Nginx, HAProxy, Squid, Envoy) can run as either. The distinction is entirely about **position and allegiance**: which side of the conversation the proxy represents, and whose identity it hides.

This document goes beyond the basic definitions and covers architecture, request lifecycle, security posture, performance implications, and production-grade configuration for both.

2. Forward Proxy

2.1 Definition and Position

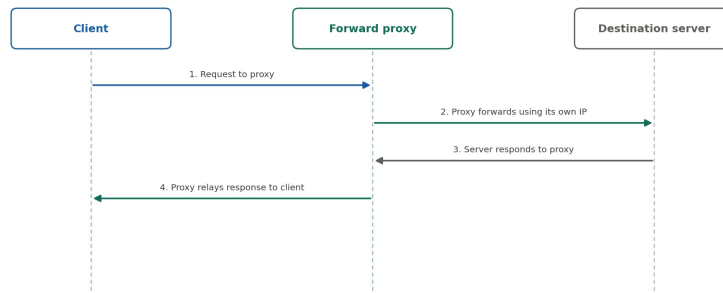
A forward proxy sits **in front of a group of clients** and mediates their outbound requests to the wider internet. It is deployed and controlled by (or on behalf of) the client side — a company, an ISP, an individual running a VPN client.



Forward proxy architecture

The diagram above reflects a realistic deployment, not a toy example: clients don't hit the internet directly — they pass through a forward proxy that applies an access control list, an optional caching layer, and a firewall boundary, all before traffic leaves the corporate network. Every external server in this picture receives connections that originate from the proxy's IP, never the individual client's.

2.2 Request Lifecycle



Forward proxy sequence

1. The client sends its request to the forward proxy — this is explicit; the client (or its OS/browser) is configured to know the proxy exists.
2. The proxy evaluates policy: is this destination allowed? Is a cached copy available? Should this be logged?
3. If forwarding is required, the proxy opens its own connection to the destination server, using its own source IP.
4. The destination server responds — to the proxy, not to the client.
5. The proxy relays (and may modify, cache, or scan) the response before returning it to the client.

2.3 Deployment Patterns Beyond the Basics

- **Transparent forward proxy** — network-level interception (via routing rules or WCCP) with no client-side configuration required. The client doesn't know a proxy is involved at all, unlike a classic explicit proxy.
- **Authenticating proxy** — requires per-user credentials before forwarding, common in enterprise environments needing per-user audit trails rather than just per-IP logs.
- **Proxy chaining** — a request can hop through multiple forward proxies (e.g. a Tor circuit), each one only aware of the hop immediately before and after it.
- **Split-tunnel vs full-tunnel VPN** — a modern variant of the forward-proxy concept, where only some traffic classes are routed through the proxy/tunnel and the rest goes direct.

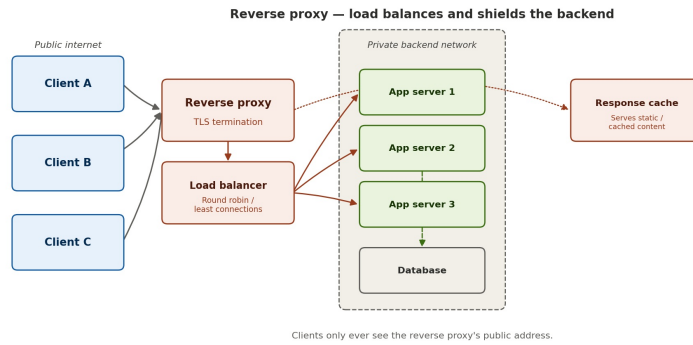
2.4 Security and Operational Considerations

- A forward proxy is a **single point of policy enforcement** — if it goes down and fails open, outbound filtering is bypassed; if it fails closed, all outbound traffic halts. Design the failure mode deliberately.
- TLS-inspecting forward proxies (common in enterprise DLP setups) terminate and re-originate HTTPS connections, which requires distributing an internal CA certificate to every client — a frequent source of certificate-trust support tickets.
- Logs at the forward proxy are a high-value target: they map internal users to every external destination they've contacted. Treat proxy logs with the same access control as any other sensitive audit data.

3. Reverse Proxy

3.1 Definition and Position

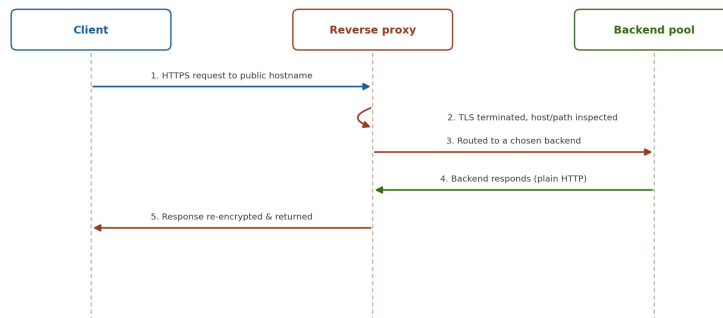
A reverse proxy sits **in front of one or more backend servers** and is deployed and controlled by the server/service operator. Clients believe they are talking directly to the destination; in reality, every request is intercepted, inspected, and routed by the proxy.



Reverse proxy architecture

Note what this architecture actually buys an operator, beyond “just another hop”: TLS termination happens once, at the edge, instead of being configured on every backend instance. The load balancer distributes load using a real algorithm (round robin, least-connections, or others), not DNS round robin, which cannot react to backend health. The backend network is private — app servers are not directly reachable from the internet, and a response cache can serve repeat requests without touching the backend fleet at all.

3.2 Request Lifecycle



Reverse proxy sequence

1. The client sends an HTTPS request to what it believes is the destination — really the reverse proxy’s public IP/hostname.
2. TLS is terminated at the proxy; the `Host` header and request path are inspected to decide how to route.
3. The proxy selects a backend using its load-balancing algorithm (and any health checks currently in effect) and forwards the request, typically over plain HTTP inside the private network.
4. The backend processes the request and responds to the proxy.
5. The proxy re-applies TLS (or the connection to the client was already TLS-terminated at the edge) and returns the response, often adding compression or cache headers.

3.3 Deployment Patterns Beyond the Basics

- **Path-based routing** — `/api/*` → microservice A, `/static/*` → a CDN-backed bucket, `/` → the main app, all behind one hostname.
- **Canary / blue-green routing** — the proxy sends a small percentage of traffic to a new backend version based on weighted rules, before a full cutover.
- **Sticky sessions** — routing a given client consistently to the same backend (via cookie or IP hash) when backend state isn’t shared, e.g. in-memory sessions.
- **Circuit breaking** — the proxy stops routing to a backend that’s failing health checks, rather than continuing to send it traffic.
- **API gateway** — a reverse proxy extended with authentication, rate limiting, and request/response transformation, sitting in front of many microservices.

3.4 Security and Operational Considerations

- The reverse proxy becomes the **single most attacked component** in the architecture, since it's the only publicly reachable entry point — hardening it (rate limiting, WAF rules, strict header validation) matters more than hardening any individual backend.
- Because backend servers only ever see traffic from the proxy's internal IP, they lose visibility into the real client IP unless the proxy explicitly forwards it (X-Forwarded-For) — and backends must not blindly trust that header unless they can guarantee the proxy is the only path in, or it becomes spoofable.
- A misconfigured reverse proxy is a classic vector for **request smuggling** — where proxy and backend disagree on where one HTTP request ends and the next begins. Keeping proxy and backend HTTP parsing behavior aligned (or using HTTP/2 end-to-end) mitigates this.

4. Side-by-Side Comparison

Dimension	Forward Proxy	Reverse Proxy
Positioned in front of	Clients	Servers
Deployed/controlled by	Client-side organization or user	Server-side operator
Client awareness	Usually explicit (configured)	Usually invisible
Identity hidden	The client, from the destination	The backend server, from the client
Primary motivations	Privacy, outbound policy, filtering, caching outbound content	Load balancing, TLS termination, routing, shielding backend, edge caching
Attack surface concern	Exfiltration via the proxy, TLS-inspection trust issues	Single public entry point, request smuggling, header spoofing
Common software	Squid, enterprise VPN gateways, Tor	Nginx, HAProxy, Envoy, Traefik
Failure mode to design for	Fail-open vs fail-closed outbound policy	Health-check-driven failover across backend pool

5. Production-Grade Nginx Reverse Proxy Configuration

A minimal `proxy_pass` block is a starting point, not a production config. Below is a more complete setup with load balancing, health-aware failover, caching, rate limiting, and correct header propagation.

```
# Upstream pool with health-aware load balancing
upstream backend_pool {
    least_conn;
    server 10.0.1.10:3000 max_fails=3 fail_timeout=30s;
    server 10.0.1.11:3000 max_fails=3 fail_timeout=30s;
    server 10.0.1.12:3000 backup;    # only used if the primaries are
down
    keepalive 32;                    # reuse connections to backends
}

# Rate limiting zone – 10 requests/sec per client IP
```

```

limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;

# Cache zone for proxied responses
proxy_cache_path /var/cache/nginx levels=1:2
keys_zone=proxy_cache:10m max_size=1g inactive=60m;

server {
    listen 443 ssl http2;
    server_name example.com;

    ssl_certificate      /etc/nginx/ssl/example.com.crt;
    ssl_certificate_key  /etc/nginx/ssl/example.com.key;
    ssl_protocols TLSv1.2 TLSv1.3;

    location /api/ {
        limit_req zone=api_limit burst=20 nodelay;

        proxy_pass http://backend_pool;
        proxy_http_version 1.1;
        proxy_set_header Connection "";          # required for
keepalive to upstream

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_connect_timeout 5s;
        proxy_read_timeout 30s;

        proxy_cache proxy_cache;
        proxy_cache_valid 200 10m;
        proxy_cache_use_stale error timeout updating http_500
http_502 http_503;
        add_header X-Cache-Status $upstream_cache_status;
    }

    location /static/ {
        root /var/www;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }
}

# Redirect plain HTTP to HTTPS
server {
    listen 80;
    server_name example.com;
    return 301 https://$host$request_uri;
}

```

What's doing real work here, beyond a beginner config: - `least_conn` + `max_fails`/`fail_timeout` — traffic favors the backend with fewer active connections and automatically routes around one that's failing, instead of blind round robin. - `backup` — a standby server only receives traffic if every primary is down. - `keepalive 32` + `proxy_http_version 1.1` — reuses TCP connections to the backend pool instead of opening a new one per request, which matters a lot under load. - `limit_req zone` — protects the backend from being overwhelmed by any single client, at the edge, before the request ever reaches app code. - `proxy_cache` with `proxy_cache_use_stale` — serves a stale cached copy if the backend is temporarily erroring or slow, instead of failing the request outright.

6. Production-Grade Forward Proxy Configuration (Squid)

A more realistic Squid forward-proxy configuration than a bare allow-all rule — this one authenticates users, restricts destinations by category, and caches aggressively:

```
http_port 3128

# Basic authentication against an external helper
auth_param basic program /usr/lib/squid/basic_ncsa_auth
/etc/squid/passwords
auth_param basic realm Corporate Proxy
acl authenticated proxy_auth REQUIRED

# Only allow traffic from the internal network, and only if
authenticated
acl localnet src 192.168.1.0/24
http_access allow localnet authenticated
http_access deny all

# Block a category of destinations via an external domain list
acl blocked_sites dstdomain "/etc/squid/blocked_domains.txt"
http_access deny blocked_sites

# Cache tuning
cache_dir ufs /var/spool/squid 10000 16 256
maximum_object_size 50 MB
cache_mem 512 MB
```

This forces every outbound request through authentication and category filtering before it's allowed to leave the network — a realistic enterprise policy, not just a pass-through relay.

7. Choosing Between Them (and When You Need Both)

Most production web architectures run **both** simultaneously, on different sides of the same request path:

- A reverse proxy sits in front of your servers (load balancing, TLS termination, routing).
- Employees inside the company accessing the internet may go through a separate forward proxy (outbound filtering, DLP).

These are not competing choices — they solve problems on opposite sides of the network boundary, and a mature architecture typically has both deployed independently.

8. Summary

A forward proxy represents and protects the **client**, standing between internal users and the outside internet. A reverse proxy represents and protects the **server**, standing between the outside internet and internal backend infrastructure. Same underlying mechanism — relaying traffic through an intermediary — but opposite allegiance, opposite attack surface, and opposite set of problems solved.